Engenharia de um cliente HTTP sem navegador para o protocolo privado de conversação do DeepSeek Web

Relatório técnico - Versão 1.1 | 24 de outubro de 2026

Projeto: DeepSeek Web HTTP Client

Natureza: estudo de implementação e avaliação experimental, sem afiliação ao DeepSeek.

Resumo

Este trabalho documenta a construção e validação de um cliente que se comunica diretamente por HTTP com o serviço web de conversação do DeepSeek, sem inicialização de navegador, automação visual ou motor de renderização. A implementação Python reproduz criação de sessão, prova de trabalho (proof-of-work, PoW), envio autenticado de prompts, upload multipart de imagens e documentos, consulta assíncrona de processamento, associação de arquivos ao prompt, leitura de Server-Sent Events (SSE) e continuação automática de respostas. O PoW inclui solucionador C e alternativa Python. Dezesseis testes automatizados locais foram aprovados. Em geração remota longa, 38.391 caracteres emitidos coincidiram exatamente com o histórico do servidor, com uma continuação automática; no modo de raciocínio curto, texto e raciocínio também coincidiram. Testes remotos autenticados com imagem PNG e documento PDF confirmaram upload, interpretação dos anexos e resposta final. Os resultados demonstram viabilidade no momento do ensaio, sem estabelecer estabilidade do protocolo privado ou disponibilidade de autenticação inicial sem credencial preexistente.

Palavras-chave: engenharia reversa de protocolo; HTTP; DeepSeek Web; Server-Sent Events; prova de trabalho; upload de mídia; continuidade de geração; cliente headless.

1. Introdução

Aplicações de conversação acessadas por páginas web frequentemente executam tarefas essenciais de rede por endpoints HTTP internos. Esses endpoints são consumidos pela interface mas podem não corresponder à API pública comercializada pelo fornecedor. A reprodução controlada do protocolo observado permite implementar uma interface programática sem motor de navegador, reduzindo dependências de renderização e separando claramente autenticação, sessão, desafio computacional e transporte de dados.

O objetivo deste projeto foi construir um cliente Python capaz de: (i) iniciar e retomar conversas; (ii) preservar autenticação previamente concedida; (iii) satisfazer o PoW de geração e upload; (iv) enviar e referenciar mídias após processamento remoto; (v) apresentar texto e raciocínio em tempo real; (vi) continuar uma resposta quando o serviço sinalizar limite de saída; e (vii) validar resultados com histórico e conteúdo conhecido dos anexos.

O objeto técnico é **chat.deepseek.com**, não o serviço documentado de API pública em **api.deepseek.com**. As interfaces aqui descritas são observações e inferências de implementação, não contratos oficiais. A documentação pública do provedor utiliza outro endereço base [1].

2. Escopo e requisitos funcionais

O produto desenvolvido é uma biblioteca e ferramenta de linha de comando: api_client.py. Não é, nesta versão, um daemon HTTP, plugin do servidor DeepSeek, serviço compatível com o formato OpenAI nem solução de autenticação por senha.

Os requisitos verificados no desenho incluem:

1. Nenhum navegador precisa existir na máquina que executa chamadas normais.
2. Todo tráfego de geração usa HTTPS por uma sessão requests.Session.
3. Credenciais salvas são reutilizadas a partir de cofre do sistema via keyring.
4. O cliente calcula o desafio específico DeepSeekHashV1 antes da conclusão.
5. A emissão de SSE produz deltas utilizáveis em CLI, JSONL ou callbacks.
6. O fim de um segmento INCOMPLETE aciona o endpoint de continuação da **mesma mensagem**.
7. Uma nova mensagem pode reutilizar uma sessão existente com UUID e ID da mensagem pai.
8. Falhas de autenticação, PoW, rede, protocolo e limite de continuação não são convertidas em respostas de sucesso.
9. Imagens e documentos locais podem ser enviados por multipart e associados ao prompt após confirmação remota de processamento.

A importação de credencial de armazenamento local de navegadores anteriormente utilizados é **opcional e offline**. Esse utilitário lê arquivos de perfis já existentes, mas não abre processo de navegador. Se a máquina nunca recebeu sessão válida e não há bearer token do titular, a primeira autenticação não é suprida por este cliente.

3. Metodologia

3.1. Caracterização e implementação

O estudo partiu do tráfego web capturado e de artefatos gerados com o projeto Reverse API Engineer [2], seguido de inspeção e implementação incremental dos endpoints. Os campos observados foram separados em dados de sessão, prova de trabalho, configuração da geração, resposta em fluxo e conclusão. Os casos reais serviram para confrontar o comportamento programático com o histórico persistido remotamente.

A base Python usa requests para HTTP, keyring para credenciais e um parser SSE específico. O solucionador de PoW está representado tanto em C, com processo nativo invocado pelo Python, quanto em rotina Python de fallback.

3.2. Validação

A avaliação ocorreu em dois níveis: testes unitários de fluxos sintetizados de SSE, credenciais e anexos; e execuções remotas autorizadas em conta previamente autenticada. O teste de integridade comparou texto concatenado emitido durante streaming com fragments RESPONSE recuperados da sessão. Os testes multimodais submeteram imagem PNG de cor uniforme e PDF com marcador textual conhecido, verificando se a resposta correspondia ao conteúdo.

Não foi executado benchmark estatístico em múltiplos ambientes; portanto tempo de resposta, largura de banda, eficiência energética ou taxa de tokens por segundo não são objeto de afirmação quantitativa deste trabalho.

4. Arquitetura

A execução é composta pelas seguintes camadas:

- **Entrada:** CLI, código Python, prompt, anexos por caminho/bytes nomeados, flags de thinking e busca, identificadores opcionais da conversa.
- **Credenciais:** resolução de bearer, device id e dados auxiliares; leitura opcional do cofre do sistema.
- **Controle HTTP:** criação ou reutilização da sessão, negociação de PoW por rota, upload multipart, polling de arquivos, POST da mensagem, acompanhamento de códigos e erros.
- **Computação PoW:** cálculo do nonce por executável nativo, com fallback Python.
- **Transporte incremental:** recepção de eventos text/event-stream, reconstrução de patches e deduplicação.
- **Continuidade:** laço que repete a geração incompleta pela rota específica.
- **Saída:** texto em stdout, thinking opcional em stderr, eventos JSONL de arquivo/texto ou ChatResult completo com IDs de mensagens e arquivos.

O fluxo principal pode ser expresso como:

```
Obter credencial -> criar ou reutilizar sessão
-> [upload de anexos com PoW e polling SUCCESS]*
-> obter desafio PoW de completion
-> resolver nonce -> POST /chat/completion (ref_file_ids) -> consumir SSE
-> [INCOMPLETE? POST /chat/continue -> consumir SSE]*
-> FINISHED -> ChatResult / eventos
```

O projeto separa **continuação de geração** (message_id da resposta atual, sem prompt novo) de **continuação de conversa** (parent_message_id com novo prompt, no mesmo UUID).

5. Protocolo HTTP

5.1. Sessão e autenticação

O endpoint de criação de sessão é POST /api/v0/chat_session/create com corpo JSON vazio. O retorno contém chat_session.id no envelope de resposta do servidor. O cliente utiliza Authorization: Bearer ... e cabeçalhos de contexto do aplicativo web, inclusive locale, timezone, versão e identificador de dispositivo. Erros HTTP e campos code/biz_code são avaliados, pois um status HTTP 200 não garante sucesso lógico.

5.2. Prova de trabalho

Antes da geração, o cliente solicita POST /api/v0/chat/create_pow_challenge informando target_path igual a /api/v0/chat/completion. O servidor devolve dados de desafio e limite de busca.

A condição de aceitação implementada é:

```
prefixo = UTF8(salt + "_" + expire_at + "_")
encontrar o menor n inteiro em [0, difficulty):
    DeepSeekHashV1(prefixo || UTF8(n)) = challenge
```

O algoritmo DeepSeekHashV1 mantém a estrutura de sponge, padding e saída de 256 bits da família SHA-3 [3], mas utiliza a permutação Keccak-f[1600] com as rodadas **1 a 23**, omitindo a rodada 0. Isso o diferencia de SHA3-256 padronizado. O challenge é interpretado como hexadecimal de 32 bytes. O cabeçalho x-ds-pow-response transporta em Base64 um JSON com algoritmo, desafio, salt, resposta, assinatura e caminho alvo.

Uma implementação C otimizada é usada quando disponível. Há verificação de algoritmo, comprimento do desafio e dificuldade positiva, e exceção explícita se nenhum nonce aceito é encontrado.

5.3. Geração e continuação

O endpoint POST /api/v0/chat/completion recebe chat_session_id, parent_message_id, model_type, prompt, ref_file_ids, thinking_enabled, search_enabled, action e preempt; a requisição leva o cabeçalho PoW. O modelo efetivo associado a model_type=default não é garantido como fixo.

Quando uma geração encerra com INCOMPLETE, o cliente envia POST /api/v0/chat/continue com chat_session_id, message_id da resposta atual e fallback_to_resume=true. O fluxo prossegue até FINISHED ou falha explícita. Não é criado novo prompt nesse passo.

5.4. Upload de imagem e documento

Para cada anexo, o cliente obtém um desafio POST /api/v0/chat/create_pow_challenge com target_path="/api/v0/file/upload_file". Em seguida envia o arquivo por POST /api/v0/file/upload_file, campo multipart file, com cabeçalhos x-ds-pow-response, x-file-size, x-model-type e x-thinking-enabled. O envelope da resposta informa data.biz_data.id. A rota GET /api/v0/file/fetch_files?file_ids=ID é consultada periodicamente até o arquivo atingir SUCCESS. Estados transitórios como PENDING e PARSING não permitem prosseguir; falhas e expiração do prazo levantam exceção. Os IDs confirmados são enviados na lista ref_file_ids da geração.

O PoW do upload tem caminho alvo próprio e não substitui o PoW de conclusão da mensagem. O cliente aceita caminhos locais, bytes com nome de arquivo e múltiplos anexos. O conteúdo binário não é embutido como texto no prompt; fica associado ao ID remoto. Foram validados remotamente PNG e PDF, sem pressupor suporte universal a formatos.

6. Reconstrução de SSE e integridade incremental

SSE é um formato de eventos textuais cuja gramática de linhas e tipo MIME text/event-stream são definidos no HTML Living Standard [4]. O servidor observado emite eventos como ready, title, message, close e error. O conteúdo da mensagem chega como snapshots ou patches com caminho (p), operação (o) e valor (v).

O acumulador preserva um objeto response com fragments tipados. Os fragmentos RESPONSE compõem o texto visível; THINK e tipos equivalentes compõem raciocínio, sem misturar canais. Quando uma operação adiciona o fragmento RESPONSE após THINK, eventos seguintes podem omitir o caminho, exigindo que o parser mantenha o último destino de content.

Um risco identificado no desenvolvimento foi o replay de prefixos após uma chamada de continuação: o servidor pode transmitir snapshots vazios ou desatualizados antes de reconstruir o conteúdo cumulativo. Emitir cada snapshot como novo texto geraria duplicações. O acumulador foi alterado para manter, independentemente dos snapshots, os prefixos _emitted_text e _emitted_thinking. Um snapshot que ainda não contém todo o prefixo emitido é tratado como regressão temporária, não como reinício da saída. Apenas sufixos que estendem os prefixos são propagados.

Esse algoritmo favorece integridade append-only. Uma reescrita não prefixada pelo servidor não pode ser codificada perfeitamente como continuação puramente incremental sem um evento explícito de substituição; trata-se de uma limitação conhecida.

7. Evidências experimentais

Os ensaios confirmados foram:

Ensaio	Métrica observada	Resultado
Testes locais automatizados	16 testes	Aprovados
Autoteste PoW e SSE	Vetores/fixtures locais	Aprovados
Resposta longa sem thinking	38.391 caracteres de texto; 1 continuação	FINISHED
Comparação com histórico da resposta longa	38.391 no streaming e 38.391 no servidor	Igualdade exata
Resposta curta com thinking	151 caracteres de resposta; 267 de thinking	FINISHED
Comparação do caso thinking	Texto e raciocínio confrontados com histórico	Igualdade exata
Upload de PNG vermelho sintético	Resposta Vermelho, 1 arquivo	FINISHED
Upload de PDF com marcador textual	Resposta JABUTI42, 1 arquivo	FINISHED

O teste longo de thinking executado antes do reparo de deduplicação identificou replay repetido e motivou a correção. **Não houve nova validação end-to-end do cenário combinado longo + thinking + continuidade após o reparo**; seus componentes foram avaliados por testes sintéticos e execuções reais separadas. Isso delimita o alcance das conclusões.

8. Segurança e disponibilidade

O cliente não deriva nem quebra credenciais. O bearer deve vir de sessão previamente autorizada ou ser fornecido pelo titular. O cofre do sistema é usado para reutilização; --forget-credentials apaga apenas a cópia local. Cabeçalhos, cookies, WAF e políticas do servidor continuam sendo restrições reais, e mudanças neles podem invalidar a compatibilidade.

A interface privada não fornece contrato público de versão, SLA, limite de requisições ou imutabilidade. Não se infere compatibilidade OpenAI nem equivalência com a API oficial. O programa oferece upload de arquivo local, mas não listagem de conversas nem persistência de estado de sessão independente do histórico remoto. O upload transmite dados à conta do serviço, requer autorização apropriada para seu conteúdo e está sujeito a formatos e limites impostos pelo servidor. Uma execução pode falhar caso expire a autenticação, ocorra mudança de protocolo, restrição de rede ou limite de continuidade.

9. Reprodutibilidade

Na pasta do projeto:

```
python -m pip install -r requirements.txt
python api_client.py --selftest
python -m unittest -q test_media test_auto_continue test_streaming test_credential_store
python -u api_client.py --stream --thinking "Responda apenas OK."
python -u api_client.py --stream --file "foto.png" "Descreva esta imagem."
```

Os primeiros testes não precisam da rede. As chamadas de geração e anexos dependem de token válido e podem variar no conteúdo retornado. A referência de implementação, contrato de eventos, exemplos e instruções detalhadas estão em docs/ e examples/.

10. Conclusão

Os resultados demonstram que o ciclo estudado da aplicação Web DeepSeek pôde ser reproduzido por um cliente HTTP puro em Python, com sessões, PoW próprio por rota, upload multipart de imagem/documento, associação de anexos à conversa, streaming de resposta e thinking, continuidade automática e integração por callbacks/CLI. A equivalência entre texto longo emitido e histórico remoto sustenta integridade no cenário testado; a identificação da cor no PNG e do marcador no PDF demonstra processamento multimodal real nesses casos. A principal fragilidade é estrutural: o protocolo é privado e mutável, e autenticação inicial depende de credencial legítima pré-existente. Trabalhos adicionais exigiriam novas medições independentes para afirmações sobre desempenho e testes após mudanças de protocolo.

Referências

[1] DeepSeek. **Your First API Call** (documentação da API pública). https://api-docs.deepseek.com/api/responses (consulta em 24/10/2026).

[2] KALIL0321. **Reverse API Engineer** (repositório de ferramenta de inspeção e geração). https://github.com/kalil0321/reverse-api-engineer (consulta em 24/10/2026).

[3] NIST. **FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions**. 2015. https://doi.org/10.6028/NIST.FIPS.202.

[4] WHATWG. **HTML Living Standard, Section 9.2: Server-Sent Events**. https://html.spec.whatwg.org/multipage/server-sent-events.html (consulta em 24/10/2026).

[5] Projeto DeepSeek Web HTTP Client. api_client.py, credential_store.py, test_media.py, test_auto_continue.py, test_streaming.py, test_credential_store.py. Código-fonte local inspecionado em 24/10/2026.